

T16

Interface Specification for CEDA Interchange Message Format

Version 1.8.1

Version Control

Version	Date	Author/Reviser	Document Revision History
1.0.0	26/04/2023	Thawatchai B.	Initial Draft.
1.0.1	26/05/2023	Waraporn S.	Change diagram. Add format, sample value and possible value. Add sample message.
1.1.0	31/05/2023	Thawatchai B.	Update header version to 1.1 Add more M Interface definition.
1.2.0	03/07/2023	Thawatchai B.	Introduce M3 Interface for Infra Services. Update header version to 1.2 by removing DB control parameters. Add more information to M1. Optimize interface format. Change some header params availability to Optional.
1.3.0	18/07/2023	Thawatchai B.	Update header version to 1.2 to support stateful data passing in the Saga pattern.
1.4.0	07/08/2023	Thawatchai B.	Update header version to 1.3 to support business correlation parameters.
1.4.1-4	17/08/2023	Thawatchai B.	Add more information to support business use-case. Review communication definition. Support multiple identity. Change some mandatory fields on M3 to optional. Add tid backward compatibility.
1.5.0	05/09/2023	Thawatchai B.	Support Loop prevention mechanism. Remove deprecated attributes, including tid. Emphasize general time format. Fix header version to 2.0
1.5.1	05/10/2023	Waraporn S.	Fix header data type of identity field to object.
1.6.0	18/10/2023	Thawatchai B.	Support Synchronous Choreography-based Saga pattern.
1.6.1	13/02/2024	Thawatchai B.	Support HTTP method for RESTful proxy capability.
1.6.2	19/02/2024	Thawatchai B.	Add more description. Support listOf in body parameter.
1.6.3	12/03/2024	Thawatchai B.	Add userAgent.

1.7.0	02/04/2024	Thawatchai B.	Introduce parameter to support Edge computing practice. Redefined QoS to support Quality of Service in M1 and M3.
1.7.1	05/04/2024	Thawatchai B.	Add messageType for explicit type of each event. Add statusCode for event status code and support HTTP response. Deprecate returnedError and use statusCode instead.
1.7.2	17/04/2024	Thawatchai B.	Add more possible values for command in M1. Add useCaseStartTime and useCaseExpiryTime to header. Update service version to support changing of communication field, in M1 and M3, to OC. Change all useCaseXXX field in M3 to Optional.
1.8.0	19/11/2024	Thawatchai B. Ramanan K.	Add destService in CEDA header to support event filtering. Add contentType in CEDA header to support type of content. Add explicit possible values of header.communication to support the Saga pattern (request-reply). Remove returnedError permanently. Update message example.
1.8.1	03/03/2025	Thawatchai B.	Generalize the meaning of baseApiVersion and schemaVersion in CEDA header. Add tmfSpec, baseApiVersion, schemaVersion, and instanceData into M3 service.

INTRODUCTION

The following document is the specification of message format passing within CEDA (Command and Event Driven Architecture) infrastructure for MFAF (Mobile First Application First) and ESB (Enterprise Service Buses) projects. It includes an available message flow example.

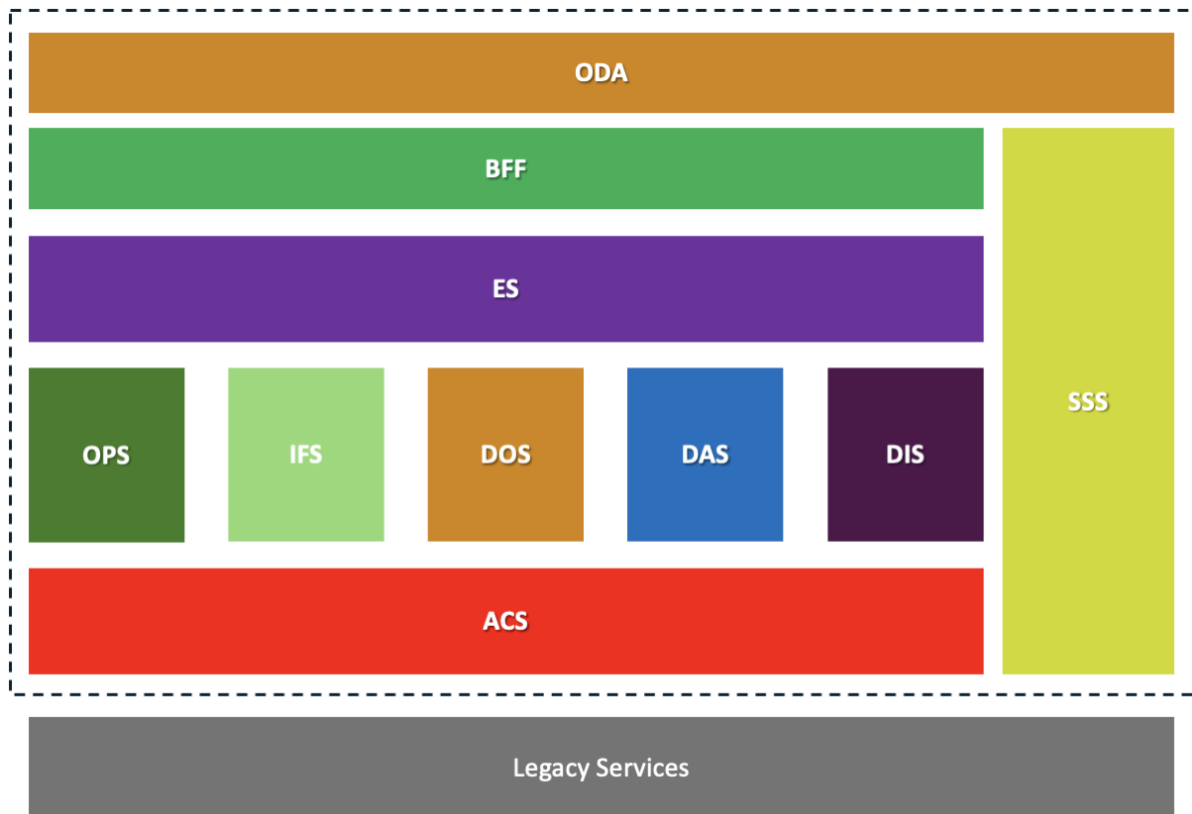
The CEDA is designed to support data connectivity and services enabling deployments based on an event-based model and utilizing cloud-native. The CEDA architecture shall leverage event-based interactions between service buses, such as Digital Service Bus, Network Service Bus, and Network Event Bus, to provide digital services to each customer facing channels, for example MFAF project for mobile user facing channel. By applied the event driven and asynchronous request/reply practices, each of service components and functions mostly communicate with each other via service bus.

In CEDA, there are so many components and functionality. Each interface or interaction between components or group of components are following the interchange message format (IMF) definition, which defined in this specification. The interchange message format is a conceptual reference format of interaction between components via shared event bus or reference point.

The group of components is categorized into the following.

1. On Device Application (ODA)
2. Backend for Frontend (BFF)
3. Event Store (ES) or Service Bus (SB)
4. Data Service (DAS)
5. Data Insight Service (DIS)
6. Domain Service (DOS)
7. Infra Service (IFS)
8. Operation Service (OPS)
9. Anti-corrupt Service (ACS)
10. Security Supplementary Service (SSS)

The big picture of group of components implemented in the CEDA is shown below.



Followings are the name of BFF, OPS, IFS component which are in the green block. This is the list of the component that are likely to be commonly used in most of the network functions.

1. Mobile Device Management (MDM) is the device administration of mobile applications such as myAIS. MDM is implemented to provide application and infrastructure capabilities and controls.
2. Mobile Backend (MBE) is the backend service for mobile application. MBE is implemented as a websocket gateway for mobile application on device. As the gateway, the MBE also support authentication and authorization for security perspective.
3. API Management (APIM) is the API gateway. APIM is implemented to manage API lifecycle.
4. Mobile Federated Identity Service (MFIS) is the identity service for any frontend channel to access backend service capability. MFIS is also implemented to provide authentication and authorization service for the applications.
5. File Management Service (FMS) is the service to allow application to transfer files between frontend and backend services.
6. Content Delivery Network (CDN) is the static content hosting service which normally called CDN (Content Delivery Network). The CDN is usually Cloud service to offers a global solution for rapidly delivering content.
7. Ingress Access Control Service (IACS) is the access control service for incoming traffic and delivery particular message to destination services.
8. Egress Access Control Service (EACS) is the access control service for outgoing traffic to mobile application.

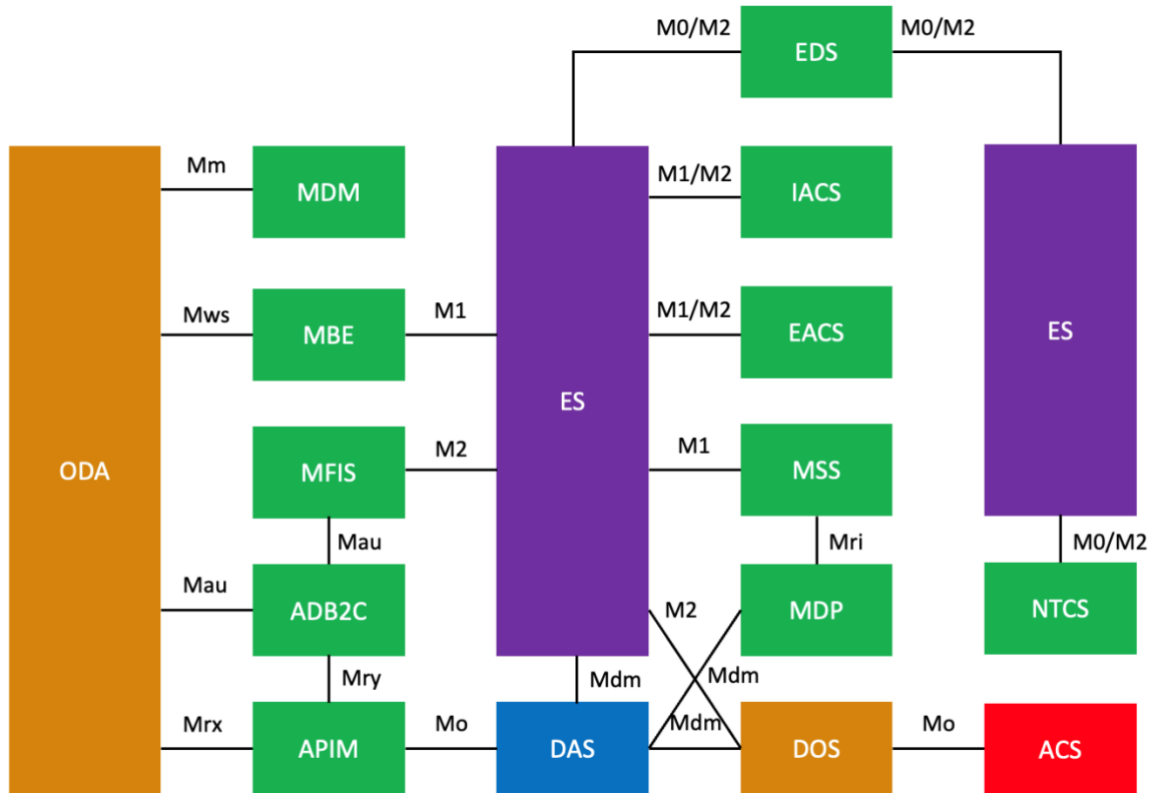
9. Mobile Supplementary Service (MSS) is the supplementary, non-business logic, service for mobile applications. MSS is implemented to leverage common capability to mobile application such as pre-initialized customer data transfer, customer activity.
10. Mobile Data Provider (MDP) is one of supplementary service to provide pre-initialized customer data transfer to requesting mobile application.
11. Mobile Data Synchronizer (MDS) is one of supplementary service to provide customer data synchronization as an Event data from Service Bus to mobile application.
12. MyAIS API Service (MAS) is the backward compatible API service for legacy MyAIS system. MAS shall be removed once the legacy MyAIS is terminated.
13. MyAIS WebView Service (MWS) is the backward compatible WebView service for legacy MyAIS system. MWS shall be removed once the legacy MyAIS is terminated.
14. API Adaptor Service (ADS) is the backward compatible API service for legacy BSS system. ADS is also implemented in Service Bus integration practice to loosen existing IT architecture.
15. Non TMF Conversion Service (NTCS) is the miscellaneous (infra) service to convert Non TMF command/event to standard TMF command/event and vice versa.
16. Event Duplicator Service (EDS) is the miscellaneous (infra) service to duplicate command or event topic across different Service Busses. EDS supports both North-South and East-West topic duplication.

Followings are the name of DAS and DIS component which are in the blue block. This is the list of the component that are likely to be commonly used in most of the data service and insight functions.

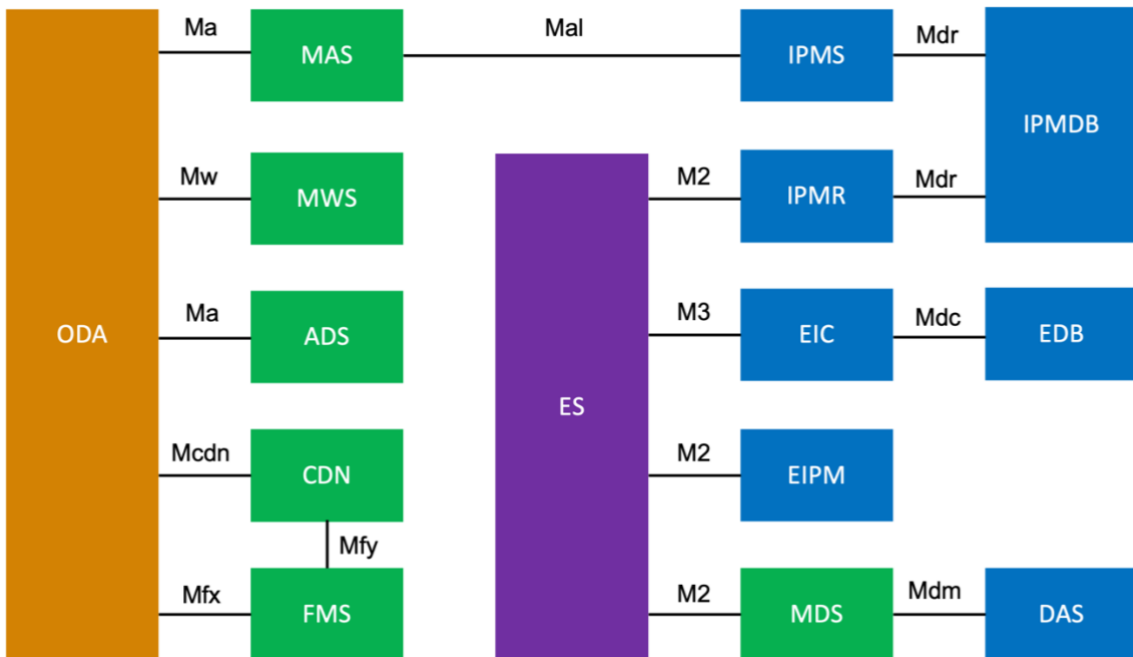
1. Identity Pixel Management Service (IPMS) is one of data service which provides full relational identity data for domain services.
2. Identity Pixel Management Retainer (IPMR) is one of data service which monitors changed event in service bus and perform data consolidation.
3. Identity Pixel Management Database (IPMDB) is relational database for IPMS.
4. Experience Insight Capturer (EIC) is data insight service which captures customer journey for business data analytics.
5. Experience Insight Persona and Preference Manager (EIPM) is data insight service which provides customer persona and preference capability.
6. Event Database (EDB) is wide-column database for Experience Insight service.

The infra/operation services and reference point can be shown in the picture below.

Classification: Internal



The infra/operation services support basic functionality.



The infra/operation services support operation functionality.

Here is a list of reference point interface format.

1. Mm interface exposed by MDM following TMF702.
2. Mau interface exposed by ADB2C and MFIS following OAuth2 and OIDC.
3. Mws interface exposed by MBE following WebSocket.io.
4. Mrx interface exposed by APIM for API with Authorization pattern.

Classification: Internal

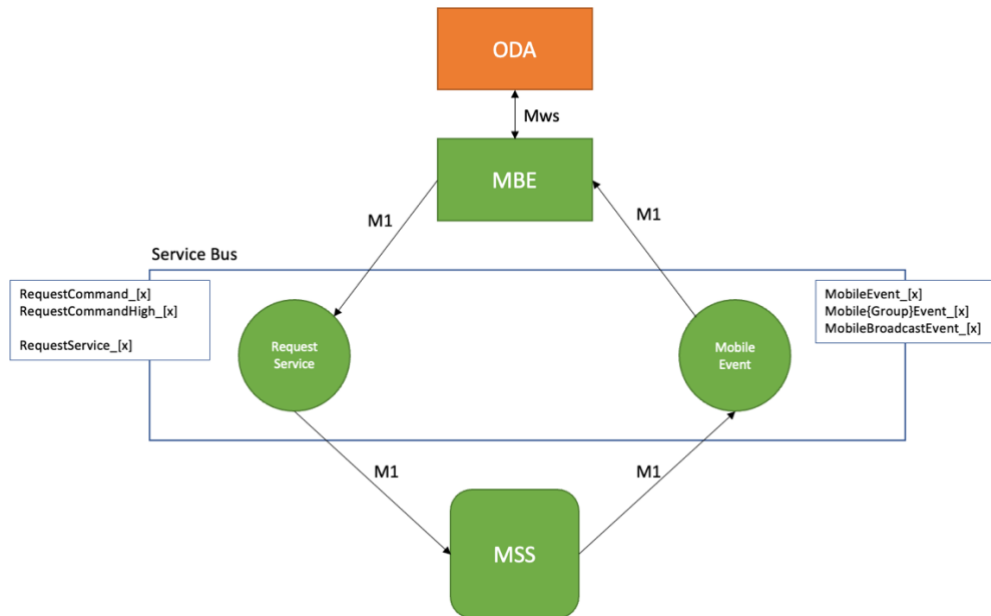
5. Mry interface exposed by ADB2C to provide token validation for APIM.
6. Mo interface for TMF openAPI standard.
7. Ma interface for API exposed by MAS, and ADS.
8. Mw interface for WebView exposed by MWS.
9. Mcdn interface exposed by CDN for media caching service.
10. Mfx interface exposed by FMS for file management.
11. Mfy interface between CDN and FMS internally manage media file.
12. Mal interface exposed by IPMS for API asset list authorization.
13. M0 interface for unstructured format in Kafka topic.
14. M1 interface for access control message format in Kafka topic.
15. M2 interface for domain interchanging message format in Kafka topic.
16. M3 interface for infra interchanging message format in Kafka topic.
17. M4 interface for operation control message format in Kafka topic. (*Early state*)
18. M5 interface for insight data message format in Kafka topic. (*Early state*)
19. M6 interface for error message format in Kafka topic. (*Early state?*)
20. Mdm interface exposed by DAS for mongoDB database access.
21. Mdr interface exposed by IMPDB for PostgreSQL database access.
22. Mdc interface exposed by EDB for Cassandra database access.

Classification: Internal

SERVICE BUS USECASE

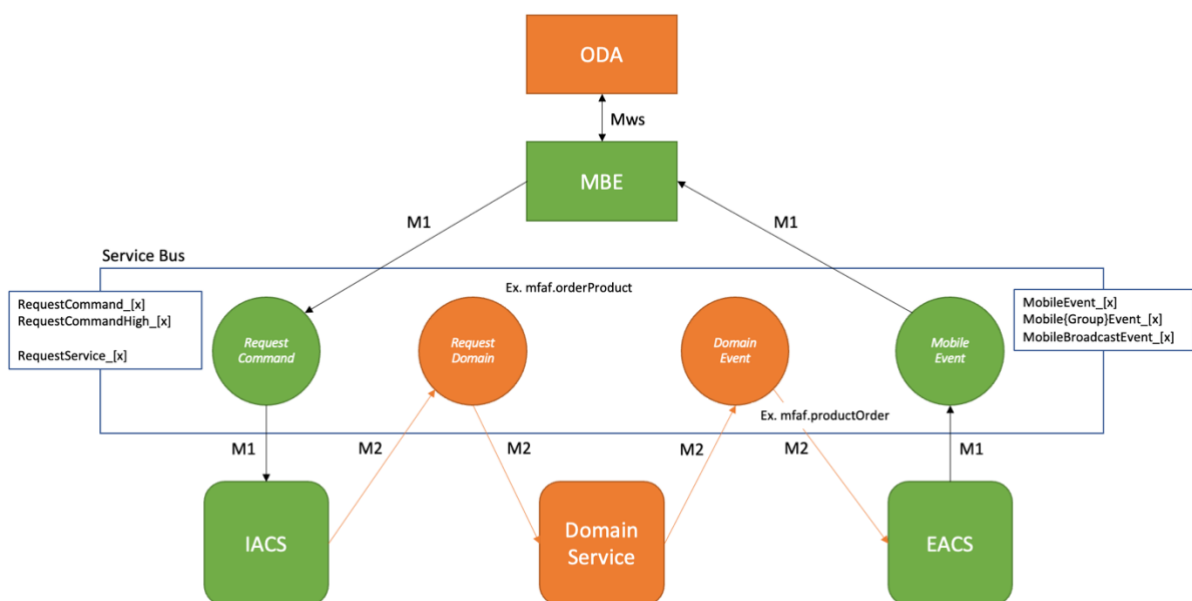
To understand the interaction between applications or services who connect to the service buses.

1. The supplementary service for ODA, or another facing channel system, usecase.



The MSS (Mobile Supplementary Service) is used for supplementary service from ODA such as mobile data initialization, data synchronization, or event log tracing service.

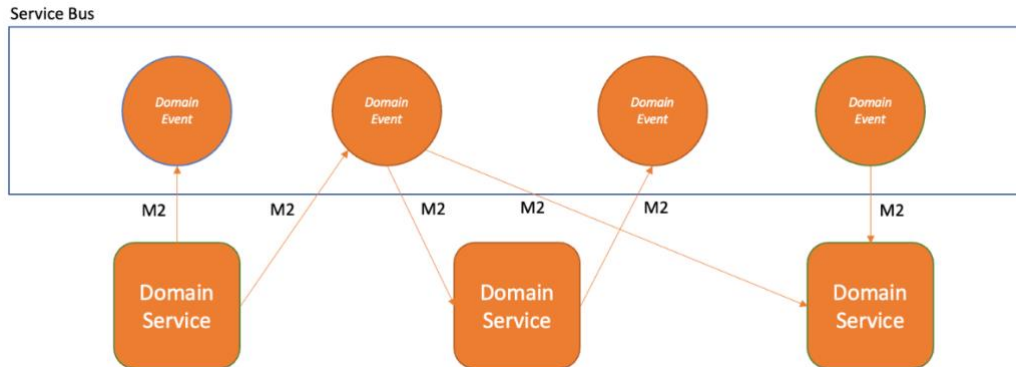
2. The business processing service for ODA, or another facing channel system, usecase.



Classification: Internal

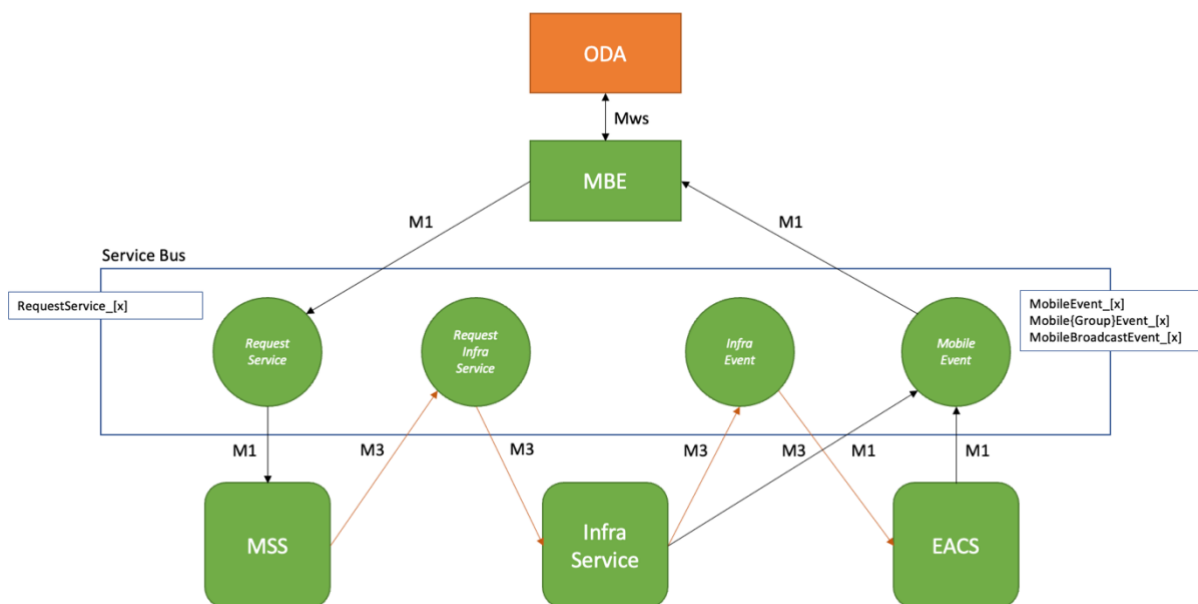
For the business processing service for ODA usecase, there will be IACS (Ingress Access Control Service) and EACS (Egress Access Control Service), which are one of infra services, to control incoming command from ODA (On Device Application) and outgoing event.

3. The inter domain communication usecase.



The domain service itself can independently produce its event by using M2 reference point. The M2 message format supports both TMF and non-TMF data passing between each domain.

4. The infra service communication usecase.



In case of infra services who also use the service bus to extent infra capabilities for customer facing channel. The chaining event shall be generated by MSS. The M3 format shall be used by infra services for specify control information.

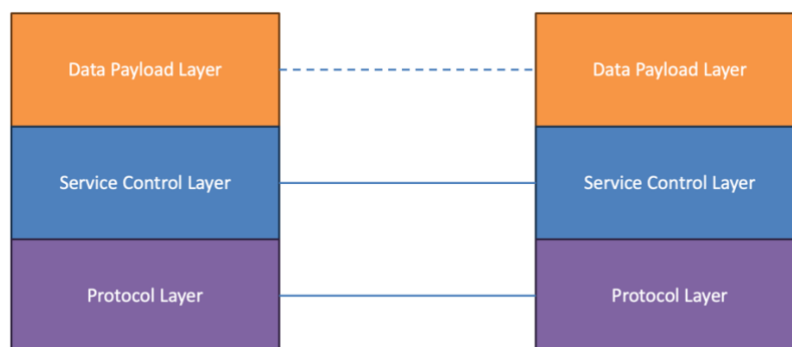
SERVICE BUS PRINCIPLE AND MECHANISM

In addition to service bus use cases which are system component and integration mechanism. The following practices in this section show the usage of Mxx interface parameters to control service behavior in term of engineering perspective.

1. CEDA version concept

There are the different version format definitions within the service bus interface (Mxx), which are represented to different levels of communication layers. The communication layers can be categorized in the following layers, also shown in the figure below.

1. Protocol Layer.
2. Service Control Layer.
3. Data Payload Layer.



The protocol layer is used to provide protocol specific information from BFF to the service bus, normally in M1 format. The service control layer is control data used by every CEDA services in order to enable CEDA service capabilities. Lastly, the data payload layer is opaque data for corresponding services.

The version numbering in protocol layer is defined in decimal number such as “1,” “2,” ... so on. Whereas the version number in service control layer is defined in dotted-decimal number and divided into 2 decimal number such as “1.0,” “1.1,” “2.0,” ... so on where the first former number is the major version and later number is the changes. It is noted that there is no fix bug number in this number.

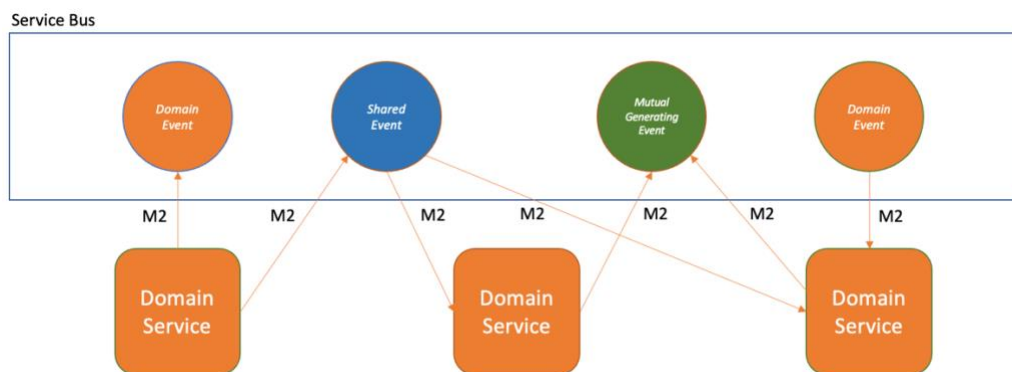
Finally, the version numbering in data payload layer is not strictly defined in this specification. However, it's recommended to use the semantic versioning. The semantic versioning (aka SemVer) is a widely-adopted version scheme that encodes a version by a three-part version number such as “1.0.0,” “1.1.0,” “1.1.1,” “2.0.0,” ... so on which represent to (Major.Minor.Patch) format.

It'd be a good practice to perform version numbering format validation. However, the value of version should not be validated, since the CEDA services can support different version of interface specification.

2. Loop prevention

Within the service bus, there are many interexchange topics or events that are produced or consumed by domain services. This chaotic scenario makes service bus to be very extreme high complex. To simplify this, the topic and event in service bus can be naturally categorized in the following type, also shown in the figure below.

4. Dedicated Event.
5. Shared Event.
6. Mutual Generating Event.



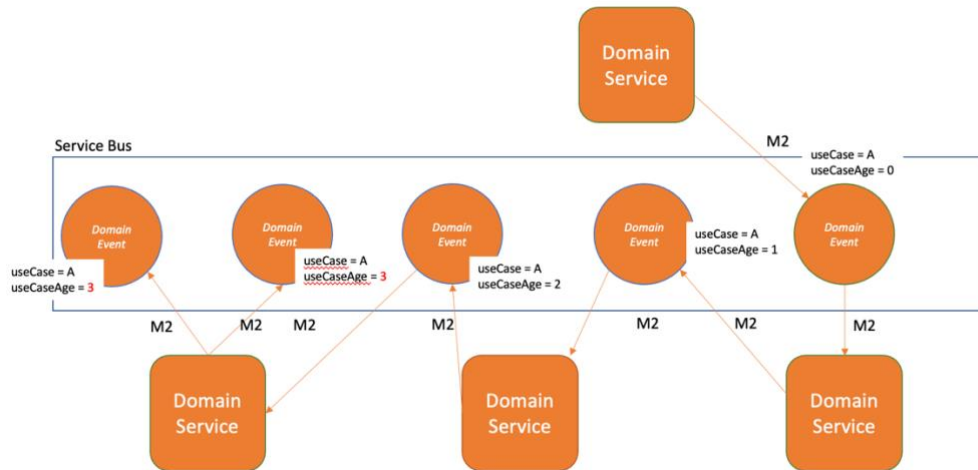
For the mutual generating event, the domain services shall set its name in “header.from” parameter in order to monitor and identify the service who generate that event. While the shared event which is consumed by domain service, it is highly recommended to use “header.orgService” parameter to substantiate whether shared event is related their own service scope.

There are very specific criteria to consider the use of “orgService”

1. That topic must be shared event.
2. That shared topic is used as a response event in synchronous choreography saga pattern. (for more info about saga pattern <https://microservices.io/patterns/data/saga.html>)

The frightened behavior of traffic loop is a number of duplicated topics in service bus which makes very high infrastructure resource consumption and finally broken through the infra capability, deny of service situation. In this case, the “header.useCaseAge” parameter is added in the interface to monitor and control this situation. The following figure shown the increment of age in service bus.

Classification: Internal

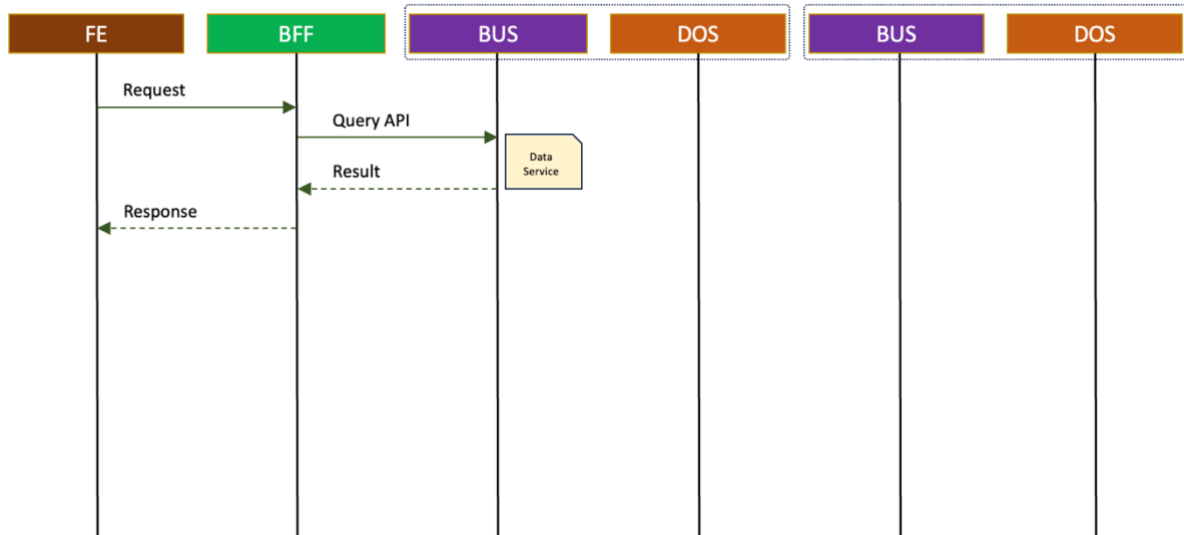


Topic or UseCase Age mechanism is consensually implemented by every single programmable system component corresponding to business use case. All domain services are responsible to verify if the age of use case exceeds an expected threshold of each business use case.

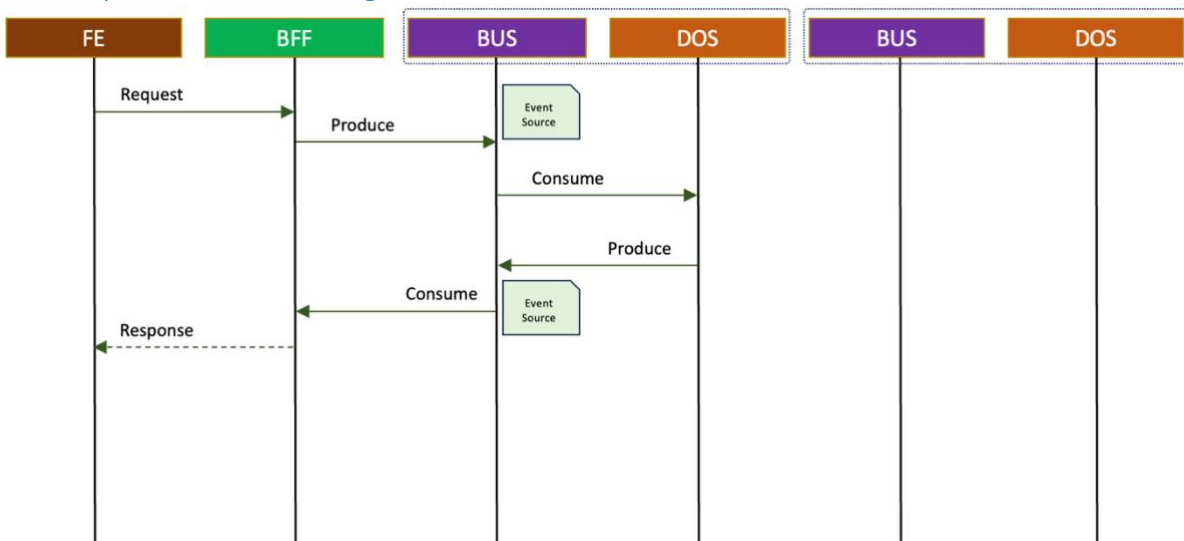
E2E SCENARIO

The usecase of service bus can be applied into E2E scenario as the following. The E2E scenario shall be focus on business process which is in particular domain services.

1. Query persistent data on bus

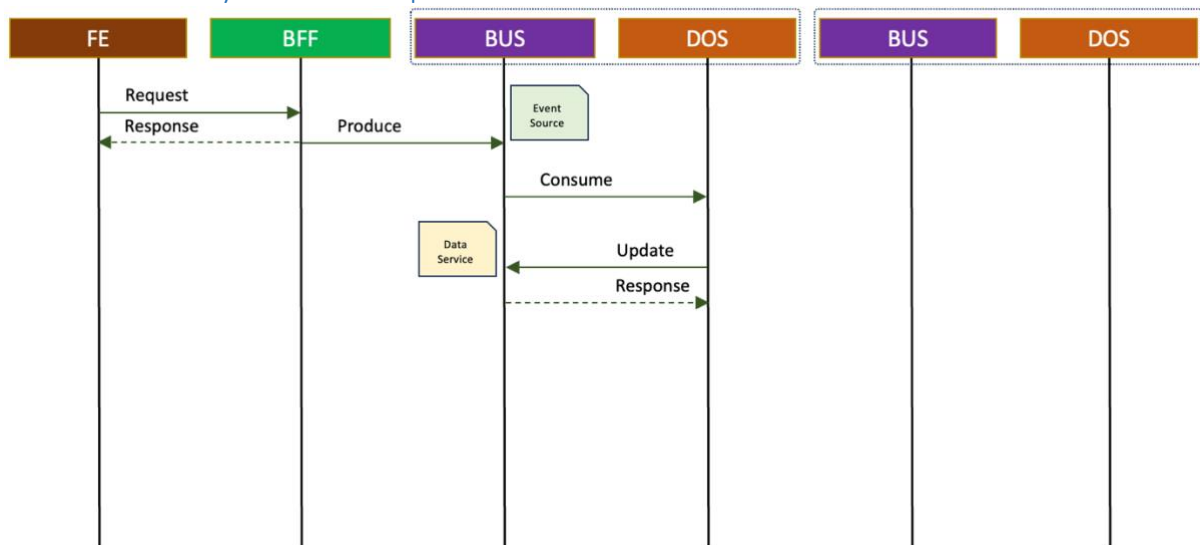


2. Query data with blocking read

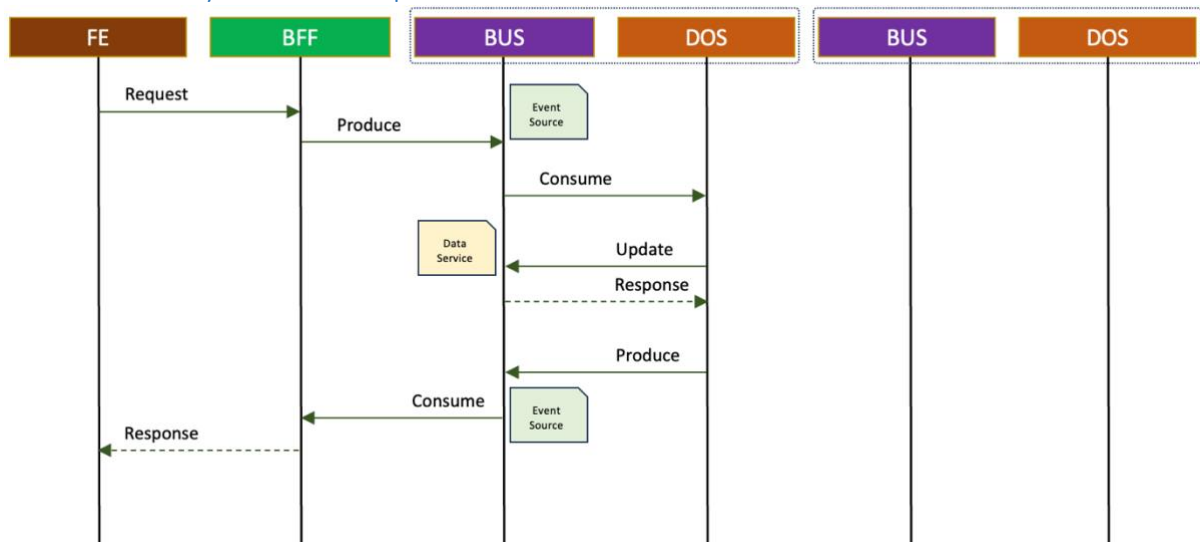


Classification: Internal

3. Command asynchronous update data

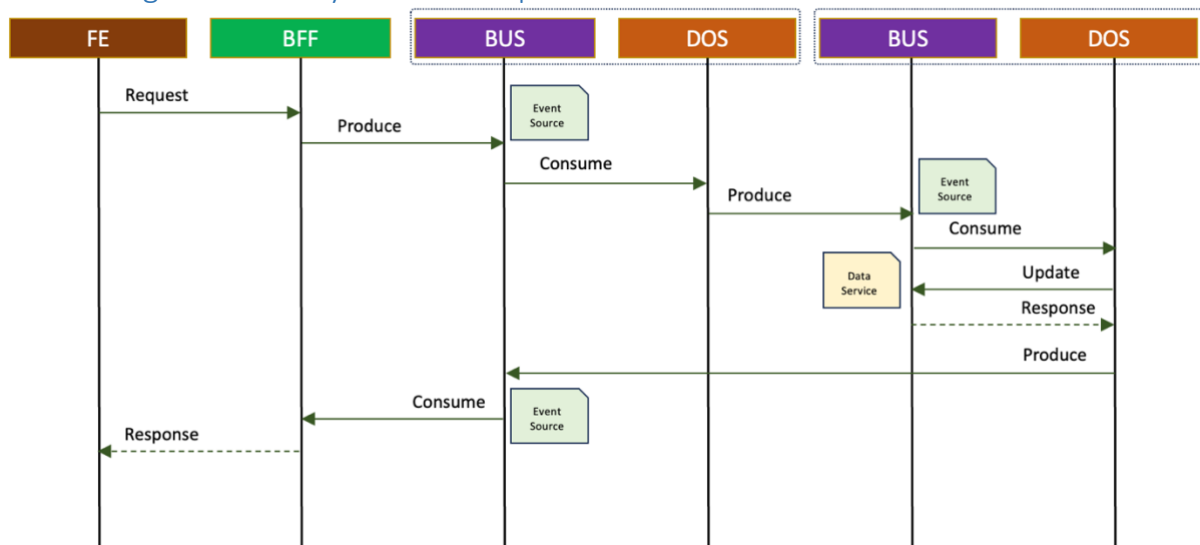


4. Command synchronous update data

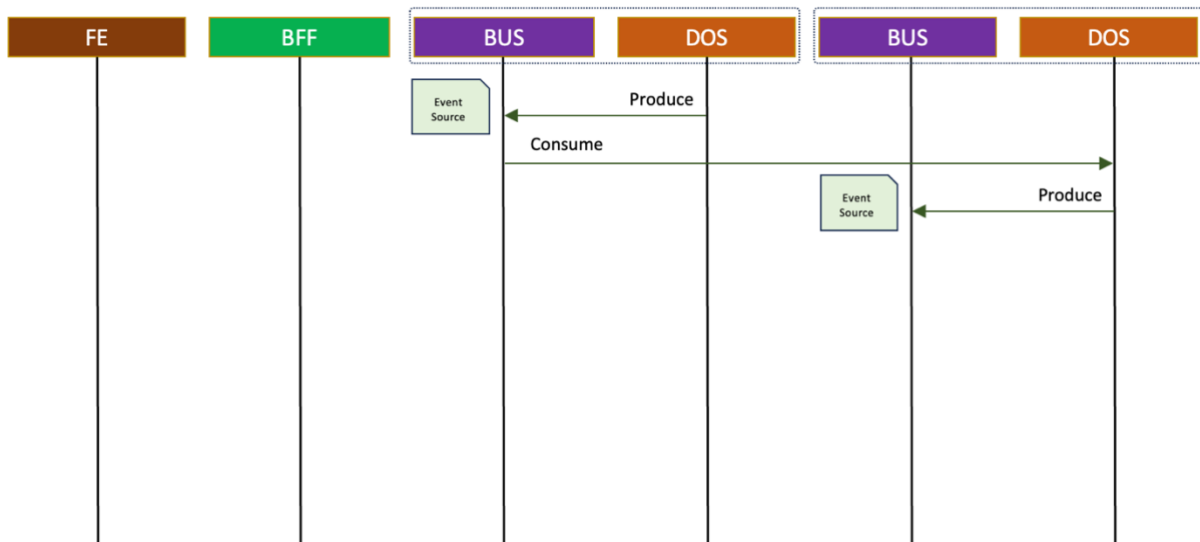


Classification: Internal

5. Chaining command synchronous update data

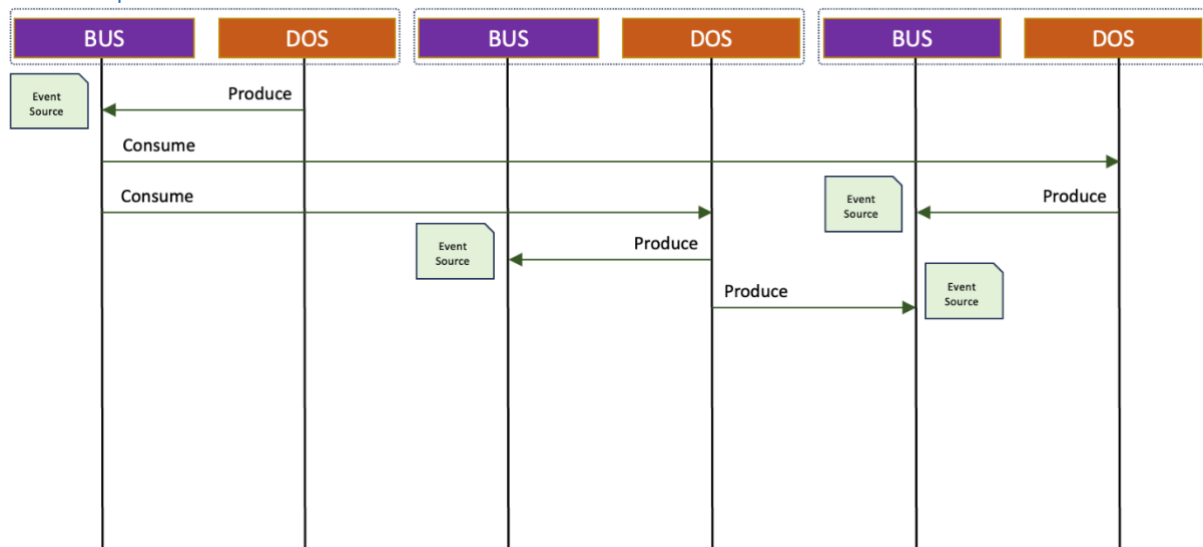


6. Inter domain communication

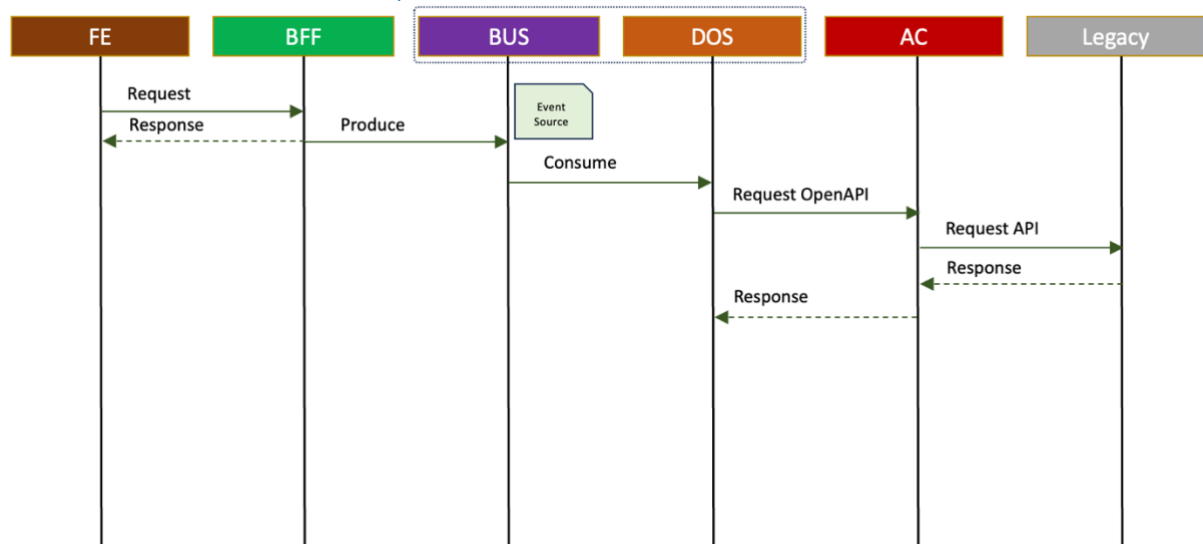


Classification: Internal

7. Multiple service bus communication



8. Command and Anti-corruption



MESSAGE FORMAT

This format shall be used by infra services to control incoming and outgoing messages.

COMMON OBJECTS

This common object format shall be used in M1 and M2 parameter.

Header Object Parameter

header				
Name	M/O /OC	Type	Description	Default value
header	M	Object	Control parameters	N/A
version	M	String	Version of header. Fixed value: 6.0	"6.0"
timestamp	M	String	Sending timestamp with timezone (ISO format). Format: ^(\d{4})-(\d{2})-(\d{2})T(\d{2}):(\d{2}):(\d{2})(?:\.\d*)?((?!(\d{2}):(\d{2}) Z)?)\$ Sample value: 2023-05-04T14:51:06.157Z	""
orgService	M	String	Originating service who initials message i.e., "party", "payment", or "product.product." This used for service correlation in the Saga pattern.	""
destService	O	String	Specific destinating service who should be received the message i.e., "party", "payment", or "product.product." This could be used for explicit event filtering when multiple services subscribed at the same event topic and producer knows exactly what the target is.	""
scope	M	String	Scope provided by FE or BFF to support Edge computing. The value shall be in dot format to show subsystem hierarchy.	"global"

from	M	String	Producer or sender who produce message to service bus.	""
channel	M	String	Front-end facing channel name.	""
broker	M	String	The performance of marketing tracking for each influencer. This can be in form of UTM parameters (the combination of source, medium, campaign, term, and content) or any variants.	""
agent	M	String	The application who initiates or requests service on particular Front-end facing channel. The format of this field could follow the HTTP User-Agent format.	""
useCase	M	String	Global unique use-case name initiated by customer or system.	""
useCaseStep	M	String	Journey step of use-case.	""
useCaseAge	M	Number	The accumulated number of hops incremented by any backend services to check whether the unwanted loop occurs. It is actually number and start with 0 by service initiator.	0
useCaseStartTime	M	String	Use-case sending timestamp with timezone (ISO format). Format: ^(\d{4})-(\d{2})-(\d{2})T(\d{2}):(\d{2}):(\d{2})(?:\.\d*)?((-(\d{2}):(\d{2}) Z)?)\$ Sample value: 2023-05-04T14:51:06.157Z	""
useCaseExpiryTime	M	String	Expected time for particular use-case with timezone (ISO format). Format: ^(\d{4})-(\d{2})-(\d{2})T(\d{2}):(\d{2}):(\d{2})(?:\.\d*)?((-(\d{2}):(\d{2}) Z)?)\$ Sample value: 2023-05-04T14:51:06.157Z	""

	functionName	O	String	The function name used by different domain services to separate its related processing data.	""
	messageType	M	String	Type of message defined how data exchange between subsystems which is one of Possible Value: "command" for a command in pub/sub. "event" for an event in pub/sub. "request" for a request in RESTful. "response" for a response in RESTful.	"event"
	contentType	O	String	Type of Content which is used to indicate the media type of the body. The media type is a string sent along with the file indicating the format of the file. The format of string can follow HTTP Content-Type format.	""
	session	M	String	Session ID of service or usecase.	""
	transaction	M	String	Transaction ID of service.	""
	communication	M	String	Type of event which is one of Possible Value: "unicast" for an individual. "multicast" for a group of assets. "groupcast" for a group. "broadcast" for everyone. "unidirection" for one-way communication and is the same as unicast. "bidirection" for a request-reply topic pairing mechanism.	"unicast"
	groupTags	M	Array of String	A list of group tag of each individual.	[]
	identity	M	Object	Group of identity related to an individual with device, public or user ID.	{}

				Identity must have at least one attribute and at least one attribute value.	
	device	OC	Array of String	A list of unique device identifiers.	""
	public	OC	String	The public identifier.	""
	user	O	String	The user identifier.	""
	token	O	String	An input token which is initiated by BFF or copy from M1. It's noted that if token parameter is sent in M1.protocol, there is no need to put it in M1.header. This is done by MBE itself.	""
	statusCode	O	String	The explicit status code for an event.	""
	initUri	O	String	An input transparent URI information sent by HTTP client in special scenario.	""
	initMethod	O	String	An input method sent by HTTP client in special scenario.	""
	queryParam	O	String	A concatenated string represents input query parameters corresponding with input URI.	""
	tmfSpec	M	String	Version of TMF specification in body data. <u>Sample value:</u> TMF622 <u>Possible value:</u> TMF ⁿⁿⁿ none	"none"
	baseApiVersion	M	String	Version of open API in body data.	"none"
	schemaVersion	M	String	Version of SID or database schema related to body.	"none"
	instanceData	O	String	Base64 encoded format of instance data. (limit size, and claim check pattern). Managed by common framework.	""

Sample of header:

```
"header":{
  "version":"6.0",
  "timestamp":"2024-05-04T14:51:06.157Z",
  "orgService":"productModule",
  "scope":"global",
  "from": "CustomerOrderValidation",
  "channel":"","
  "broker":"","
  "agent":"","
  "useCase":"","
  "useCaseStep":"","
  "useCaseAge": 0,
  "useCaseStartTime":"2024-05-04T14:51:06.157Z",
  "useCaseExpiryTime":"2024-05-04T14:51:16.157Z",
  "functionName":"requestService",
  "messageType": "event",
  "session":"234709834750982345",
  "transaction":"3456987612389746",
  "communication":"unicast",
  "groupTags":[],
  "identity":{
    "device":["324897098709872103948709"]
  },
  "statusCode":"200",
  "initUri":"/productInventoryManagement/product",
  "initMethod":"GET",
  "queryParam":"filter=ABC",
  "tmfSpec":"TMF622",
  "baseApiVersion":"4.0.0",
  "schemaVersion":"1.0.0",
  "instanceData": ""
}
```

ACCESS CONTROL MESSAGE FORMAT (M1)

This format shall be used by infra services to control incoming and outgoing messages.

Parameters

Parameters					
M1					
Name		M/O/O C	Type	Description	Default value
protocol		M	Object	Protocol parameters.	N/A
	version	M	String	Version of protocol. <u>Fixed value:</u> 2	"2"
	command	M	String	Type of message which is one of Possible Value:	"message"

				“initial” is being sent when ODA connect to MBE or register new identity.user. “remove” when ODA unregister old identity.user. “message” when the message is used for normal scenario. “service” when the message is used for supplementary scenario. “ack” when MBE send the acknowledge to ODA.	
	subCommand	M	String	Service that used by MBE or MSS for example: “init1”, “init2”, “changeToken.”	“ ”
	invoke	M	String	Invoke Id of message.	“ ”
	qos	M	String	Quality of service which is one of <u>Possible Value:</u> “normal” for general services. “high” for high priority services. “low” for high priority services.	“normal”
	topic	M	String	This use for normal scenario to identify originating or destinating topic of domain services.	“ ”
	token	M	String	Login Token which is validated by MBE.	“ ”
	service	M	String	Originating service who issue the message i.e., “ODA”, “MBE”, or “EACS.”	“ ”
	locationInfo	M	Object	Location Information of customer	
	networkInfo	M	Object	Network Information of customer	
	deviceInfo	M	Object	Device Information of customer	
	diagMessage	M	String	Diagnostic message	“ ”
	header	M/O	Object	Control parameters see Header common parameter.	
	body	M/O	Object	Opaque data.	

			This is only for a single object data. If multiple objects are required, use the "listOf" under body instead.	
	listOf	O	Array of Object	List of Opaque data. [{}]

Sample of [command: message] with normal QOS message:

```
{
  "protocol": {
    "version": "2",
    "command": "message",
    "subCommand": "",
    "invoke": "12343456827685",
    "qos": "normal",
    "topic": "test.testTopic",
    "token":
"asdgjLKHGU*#@#JRLSDHFhkajh+@O#RUOSJHDFKLJHSKFYIQUWEHRKASF",
    "service": "ODA",
    "locationInfo": {},
    "networkInfo": {},
    "deviceInfo": {},
    "diagMessage": ""
  },
  "header": {
    "version": "6.0",
    "timestamp": "2023-05-04T14:51:06.157Z",
    "orgService": "productModule",
    "from": "CustomerOrderValidation",
    "channel": "web",
    "broker": "content",
    "agent": "",
    "useCase": "test",
    "useCaseStep": "4",
    "useCaseAge": 0,
    "functionName": "requestService",
    "session": "234709834750982300",
    "transaction": "3456987612389746",
    "communication": "unicast",
    "groupTags": [],
    "identity": {
      "device": "324897098709872103948709"
    },
    "statusCode": "200",
    "initUri": "/productInventoryManagement/product",
    "initMethod": "GET",
    "queryParam": "filter=ABC",
    "tmfSpec": "TMF622",
    "baseApiVersion": "4.0.0",
    "schemaVersion": "1.0.0"
    "instanceData": ""
  },
  "body": {
    "listOf": [{}]
  }
}
```


Sample of [command: ack] with normal QOS message:

```
{
  "protocol": {
    "version": "2",
    "command": "ack",
    "subCommand": "",
    "invoke": "12343456827685",
    "qos": "normal",
    "topic": "",
    "token": "",
    "service": "MBE",
    "locationInfo": {},
    "networkInfo": {},
    "deviceInfo": {},
    "diagMessage": "Token is not valid",
  }
}
```

Sample of [command: service] with subCommand:

```
{
  "protocol":{
    "version":"2",
    "command":"service",
    "subCommand":"init2",
    "invoke":"12343456827685",
    "qos":"normal",
    "topic":"mfaf.init2",
    "token":"asdglKHGU*#@#JRLSDHFhkajh+",
    "service":"ODA",
    "locationInfo":{},
    "networkInfo":{},
    "deviceInfo":{},
    "diagMessage":""
  },
  "header":{
    "version":"3.0",
    "timestamp":"2023-05-04T14:51:06.157Z",
    "orgService":"productModule",
    "scope":"global",
    "from": "CustomerOrderValidation",
    "channel":"",
    "broker":"",
    "useCase":"",
    "useCaseStep":"",
    "useCaseAge":0,
    "session":"234709834750982345",
    "transaction":"3456987612389746",
    "communication":"unicast",
    "groupTags":[],
    "identity":{
      "device":["324897098709872103948709"]
    },
    "tmfSpec":"TMF622",
    "baseApiVersion":"4.0.0",
    "schemaVersion":"1.0.0"
  },
}
```

Classification: Internal

```
"body": {  
  }  
}
```

DOMAIN INTERCHANGING MESSAGE FORMAT (M2)

This format shall be only used by domain services to exchange message between them.

Parameters

M2				
Name	M/O/O C	Type	Description	Default value
header	M	Object	Control parameters see Header common parameter.	
body	M/O	Object	Opaque data. This is only for a single object data. If multiple objects are required, use the "listOf" under body instead.	
listOf	O	Array of Object	List of Opaque data.	[{}]

Sample of message produced by CustomerOrderValidation Service:

```
{
  "header": {
    "version": "6.0",
    "timestamp": "2023-05-04T14:51:06.157Z",
    "orgService": "CustomerOrderValidation",
    "scope": "global",
    "from": "CustomerOrderValidation",
    "channel": "",
    "broker": "",
    "useCase": "",
    "useCaseStep": "",
    "useCaseAge": 0,
    "functionName": "requestOrder",
    "session": "234709834750982345",
    "transaction": "3456987612389746",
    "communication": "unicast",
    "groupTags": [],
    "identity": {
      "device": ["324897098709872103948709"],
    },
    "statusCode": "200",
    "initUri": "/customerOrderManagement/customer",
    "initMethod": "POST",
    "tmfSpec": "TMF679",
    "baseApiVersion": "4.0.0",
    "schemaVersion": "1.0.0"
  },
  "body": {
    "id": "9ffg-ze56-ed51",
```

```

    "href":
    "https://host:port/productOfferingQualificationManagement/v4/productOff
eringQualification/9ffg-ze56-ed51",
    "description": "Illustration of a POQ resource",
    "effectiveQualificationDate": "2019-05-03T09:37:23.429Z",
    "expectedPOQCompletionDate": "2019-05-03T09:37:23.429Z",
    "expirationDate": "2019-05-12T09:37:23.429Z",
    "instantSyncQualification": false,
    "productOfferingQualificationDate": "2019-05-02T09:37:23.429Z",
    "provideAlternative": true,
    "provideOnlyAvailable": true,
    "provideUnavailabilityReason": false,
    "qualificationResult": "qualified",
    "requestedPOQCompletionDate": "2019-05-03T09:37:23.429Z",
    "channel": {
        "id": "1",
        "name": "Online channel"
    },
    "note": [
        {
            "id": "1",
            "author": "Jean Pontus",
            "date": "2019-05-02T09:37:23.429Z",
            "text": "This is an illustration of the POQ resource"
        }
    ],
    "productOfferingQualificationItem": [
        {
            "id": "1",
            "expectedActivationDate": "2019-05-02T09:37:23.429Z",
            "qualificationItemResult": "qualified",
            "action": "add",
            "note": [
                {
                    "id": "2",
                    "author": "Jean Pontus",
                    "date": "2019-05-02T09:37:23.429Z",
                    "text": "home address is provided"
                }
            ],
            "product": {
                "@type": "Product",
                "relatedPlace": [
                    {
                        "id": "459-fgr-t78",
                        "href":
                        "https://host:port/geographicAddressManagement/v4/geographicAddress/459
-fgr-t78",
                        "@type": "PlaceRef",
                        "name": "Home address",
                        "role": "install address",
                        "@referredType": "geographicAddress"
                    }
                ]
            }
        }
    ],
    "productOffering": {
        "id": "142789",

```

Classification: Internal

```
        "href":  
        "https://host:port/productCatalogManagement/v4/productOffering/142789",  
        "name": "TMF Broadband Offer",  
        "@referredType": "ProductOffering"},  
        "state": "done",  
        "@type": "ProductOfferingQualificationItem"  
    }  
]  
}  
}
```

INFRA INTERCHANGING MESSAGE FORMAT (M3)

This format shall be only used by specific infra services to exchange message to perform particular engineering logic.

Parameters

M3				
Name	M/O/OC	Type	Description	Default value
service	M	Object	Service parameters.	N/A
version	M	String	Version of service. Fixed value: 4.0	"4.0"
timestamp	M	String	Sending timestamp with timezone (ISO format). Format: ^(\d{4})-(\d{2})-(\d{2})T(\d{2}):(\d{2}):(\d{2})(?:\.\d*)?((- \d{2}):(\d{2}) Z)?\$ Sample value: 2023-05-04T14:51:06.157Z	""
qos	M	String	Quality of service which is one of Possible Value: "normal" for general services. "high" for high priority services. "low" for high priority services.	"normal"
channel	O	String	Front-end facing channel name.	""
broker	O	String	The performance of marketing tracking for each influencer. This can be in form of UTM parameters (the combination of source, medium, campaign, term, and content) or any variants.	""
agent	M	String	The application who initiates or requests service on particular Front-end facing channel. The format of this field could follow the HTTP User-Agent format.	""

useCase	O	String	Global unique use-case name initiated by customer or system.	""	
useCaseStep	O	String	Journey step of use-case.	""	
useCaseAge	O	Number	The accumulated number of hops incremented by any backend services to check whether the unwanted loop occurs. It is actually number and start with 0 by service initiator.	0	
useCaseStartTime	O	String	Use-case sending timestamp with timezone (ISO format). Format: ^(\d{4})-(\d{2})-(\d{2})T(\d{2}):(\d{2}):(\d{2})(?:\.\d*)?((- \d{2}):(\d{2}) Z)?\$ Sample value: 2023-05-04T14:51:06.157Z	""	
useCaseExpiryTime	O	String	Expected time for particular use-case with timezone (ISO format). Format: ^(\d{4})-(\d{2})-(\d{2})T(\d{2}):(\d{2}):(\d{2})(?:\.\d*)?((- \d{2}):(\d{2}) Z)?\$ Sample value: 2023-05-04T14:51:06.157Z	""	
name	M	String	Name of invocation subservice.	""	
invoke	M	String	Invoke Id of message.	""	
session	M	String	Session ID of service or usecase.	""	
transaction	M	String	Transaction ID of service.	""	
token	O	String	Login Token which is validated by MBE.	""	
from	M	String	Producer or sender who produce message to service bus.		
orgService	M	String	Originating service who issue the message i.e., "ODA", "MBE", or	""	

				"EACS." This used for service correlation in the Saga pattern.	
	communication	OC	String	Type of event which is one of Possible Value: "unicast" for an individual. "multicast" for a group of assets. "groupcast" for a group. "broadcast" for everyone.	"unicast"
	groupTags	O	Array of String	A list of group tag of each individual.	[]
	identity	O	Object	Group of identity related to an individual with device, public or user ID. Identity must have at least one attribute and at least one attribute value.	{ }
	device	OC	Array of String	A list of unique device identifiers.	""
	public	OC	String	The public identifier.	""
	user	O	String	The user identifier.	""
	locationInfo	O	Object	Location Information of customer	
	networkInfo	O	Object	Network Information of customer	
	deviceInfo	O	Object	Device Information of customer	
	diagMessage	M	String	Diagnostic message	""
	tmfSpec	M	String	Version of TMF specification in body data. <u>Sample value:</u> TMF622 <u>Possible value:</u> TMFnnn none	"none"
	baseApiVersion	M	String	Version of API in body data.	"none"

Classification: Internal

	schemaVersion	M	String	Version of SID or database schema related to body.	"none"
	instanceData	O	String	Base64 encoded format of instance data. (limit size, and claim check pattern). Managed by common framework.	" "
	body	M/O	Object	Opaque data. This is only for a single object data. If multiple objects are required, use the "listOf" under body instead.	
	listOf	O	Array of Object	List of Opaque data.	[{}]

Sample of message of M3:

```
{
  "service":{
    "version":"3.0",
    "timestamp":"2024-05-04T14:51:06.157Z",
    "qos":"normal",
    "channel":"web",
    "broker":"content",
    "agent":"",
    "useCase":"test",
    "useCaseStep":"4",
    "useCaseAge":0
    "useCaseStartTime":"2024-05-04T14:51:06.157Z",
    "useCaseExpiryTime":"2024-05-04T14:51:16.157Z",
    "name":"init2",
    "invoke":"12343456827685",
    "session":"234709834750982300",
    "transaction":"234709834750982300",

"token":"asdgjLKHGU*#@#JRLSDHFhkajh+@O#RUOSJHDFKLJHSKFYIQUWEHRKASF",
    "orgService":"MBE",
    "from": "MSS",
    "communication":"broadcast",
    "groupTags":[],
    "identity":{
      "device":[]
    },
    "locationInfo":{},
    "networkInfo":{},
    "deviceInfo":{},
    "diagMessage":""
  },
  "body":{
    "message":"sample message of M3"
  }
}
```

Classification: Internal